

Arquitetura Conceitual da Solução SRAG

Chatbot + Orquestrador LangGraph — o usuário conversa em linguagem natural; o agente toma decisões dinâmicas (ReAct), escolhe tools e responde no chat ou gera relatório executivo.

1. O que é este sistema

Trata-se de um chatbot analítico de saúde pública para monitoramento de SRAG (Síndrome Respiratória Aguda Grave) no Brasil. A interface principal é o dashboard Shiny (<http://localhost:8080>), onde o analista digita mensagens no chatbot.

Por trás do chat está o LangGraphOrchestratorAgent: um agente orquestrador com tool calling dinâmico. Em cada turno ele decide se responde direto ou se chama uma ou mais ferramentas (métricas, séries, gráficos, notícias ou relatório). Não há um pipeline fixo “sempre métricas → sempre notícias”; a decisão é do agente, guiada pelo system prompt e pelos guardrails.

2. Fluxo entre chatbot e agente orquestrador

A figura abaixo mostra o caminho completo da mensagem do usuário até a resposta no chat ou no painel de relatório.

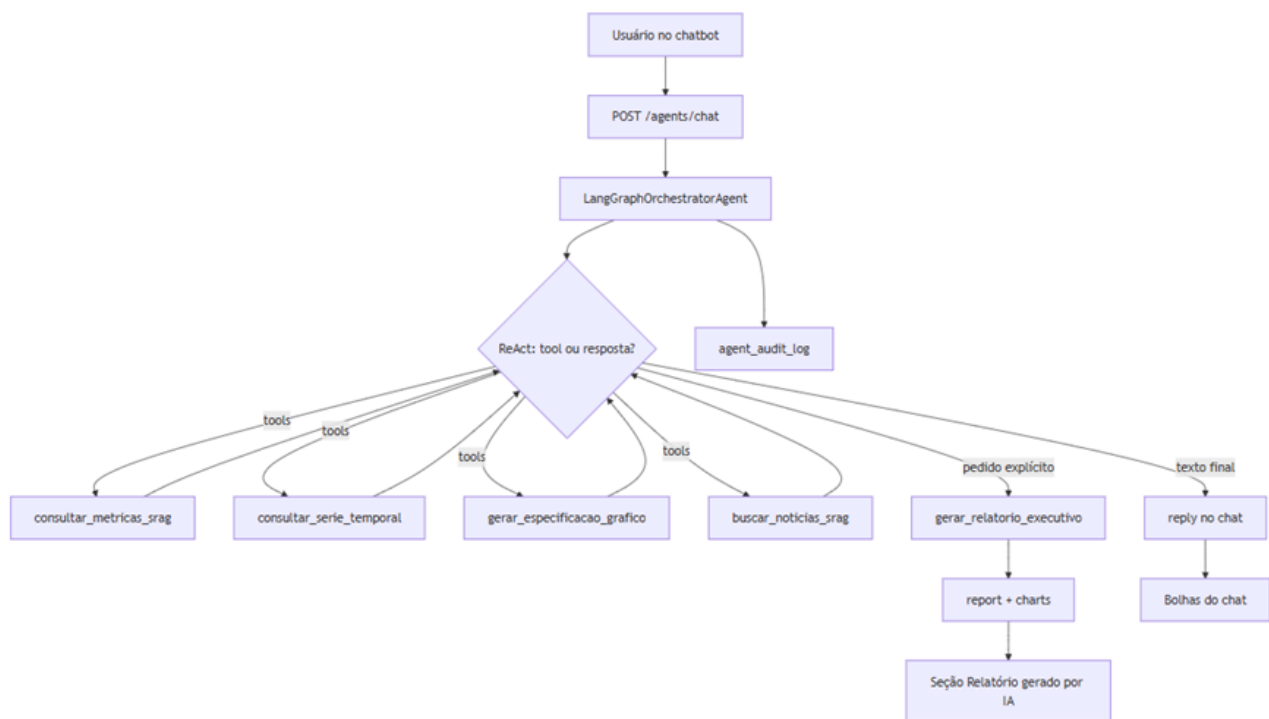


Figura — Fluxo do chatbot com LangGraphOrchestratorAgent (ReAct + tools + auditoria).

2.1 Explicação detalhada do fluxo

- 1. Usuário no chatbot — digita uma pergunta ou um pedido (ex.: “qual a mortalidade no Brasil?” ou “gere o relatório de SP”).
- 2. POST /agents/chat — o dashboard envia a mensagem (e o session_id da conversa) para a API FastAPI.
- 3. LangGraphOrchestratorAgent — recebe o turno, garante que a pipeline de dados está pronta e inicia o grafo LangGraph com memória (MemorySaver).
- 4. Auditoria em paralelo — ao final (e em erros), grava evento em agent_audit_log (tools, args, reply, duração, status).
- 5. Decisão ReAct: tool ou resposta? — este é o núcleo da autonomia do agente. A LLM analisa a mensagem + histórico e escolhe: chamar tool(s) ou emitir o texto final. Se chamar tool, recebe o resultado e volta à decisão (loop), até concluir.

- 6. Tools de análise — `consultar_metricas_srag`, `consultar_serie_temporal`, `gerar_especificacao_grafico`, `buscar_noticias_srag`. Usadas para perguntas pontuais.
- 7. Pedido explícito de relatório — somente se o usuário pedir de forma direta um relatório/resumo executivo, o agente chama `gerar_relatorio_executivo`. O texto completo + charts vão para a seção Relatório gerado por IA; no chat fica só uma confirmação breve.
- 8. Texto final → bolhas do chat — a resposta conversacional (reply) aparece nas bolhas do chatbot, com escopo e período quando métricas oficiais foram usadas.

Em resumo: o chatbot é a porta de entrada; o agente toma as decisões (quais tools, em qual ordem, se gera relatório ou só responde). O frontend apenas exibe reply e, quando houver, report + gráficos Plotly a partir de ChartSpec.

3. Componentes da solução

Frontend (Shiny)

- Chatbot no topo da página — interface conversacional
- Seção Relatório gerado por IA — texto + gráficos (não no chat)
- Sem filtro lateral de UF e sem botão “Gerar Relatório por IA”
- Escopo (UF/Brasil) e período vêm nas respostas do agente

Backend (FastAPI)

- POST `/agents/chat` — fluxo principal do chatbot
- POST `/agents/report` — relatório one-shot via API
- GET `/agents/audit*` — consulta da trilha de auditoria
- Métricas e pipeline: `/metrics/...`, `/datasets/...`

Dados

- OpenDataSUS → ETL → DuckDB `srag_notificacoes`
- Auditoria do agente em `agent_audit_log` (mesma base, tabela separada)
- Tools de métricas usam a API do projeto (não DuckDB direto)

4. Tools disponíveis (escolhidas pelo agente)

- `consultar_metricas_srag` — 4 métricas oficiais + séries
- `consultar_serie_temporal` — série diária ou mensal
- `gerar_especificacao_grafico` — ChartSpec (o dashboard desenha com Plotly)
- `buscar_noticias_srag` — Tavily Search com guardrails
- `gerar_relatorio_executivo` — relatório completo (só pedido explícito)

5. Guardrails

Os guardrails limitam o que o chatbot/agente pode fazer e o que entra no contexto. Há duas camadas principais:

5.1 Guardrails do chatbot / orquestrador (system prompt)

- Escopo restrito a SRAG / saúde respiratória no Brasil; fora disso, recusa educada.
- Não inventa números — deve usar tools oficiais da API.
- Nas respostas com métricas, informa escopo (UF ou BRASIL) e período analisado (campos `mes_anterior` → `mes_atual`).
- `gerar_relatorio_executivo` só em pedido explícito e direto de relatório/resumo executivo/painel completo.
- Perguntas pontuais (taxa, tendência, notícia) não disparam relatório.
- Nunca cola o relatório completo no chat — só confirma e aponta a seção “Relatório gerado por IA”.
- Aviso de atraso de notificação: não interpretar queda recente no fim da série como redução real sem mencionar incompleteness.

- Temperatura da LLM padrão 0 — decisões mais estáveis e reprodutíveis.

5.2 Guardrails de notícias (Tavily)

- Consulta fixa otimizada para SRAG no Brasil (`topic=news, time_range=year, max_results=5`).
- Domínios prioritários: gov.br, saude.gov.br, g1.globo.com, uol.com.br, cnnbrasil.com.br.
- Filtro pós-busca: exige Brasil/“.br” e termos SRAG/respiratório.
- Bloqueio de conteúdos inadequados (ex.: porn, sexo, violencia, racismo, politica, celebridade, guerra, crime, assassinato, terrorismo).
- Notícias são contexto complementar — não substituem dados oficiais.

5.3 Guardrails de dados e API

- Fonte oficial = API SRAG do próprio projeto.
- UF inválida → HTTP 422; falha de geração → HTTP 502.
- Resumo executivo limitado a ~4000 caracteres.
- Falha ao gravar auditoria não interrompe a resposta ao usuário.

6. Resultado para o analista

- Conversar no chatbot sobre métricas, tendências e notícias de SRAG
- Ver o agente decidir quais tools usar (visíveis no metadado da bolha)
- Pedir relatório executivo citando UF ou Brasil e vê-lo na seção dedicada
- Consultar a trilha de decisões via GET `/agents/audit`

Documento alinhado ao código atual (LangGraphOrchestratorAgent, chatbot Shiny, ChartSpec, Tavily e agent_audit_log). Figura: docs/fluxo_chatbot_orquestrador.png. Fonte textual: docs/arquitetura_solucao_srag.md e docs/agente_orquestrador.md.